

Understanding the Log Analysis Agent

Under-the-Hood Ingestion, Filtering, Hybrid RAG-LLM Logic & Technical Limits
Preparation Guide for Project Advisor Defenses

Document Specifications:

- Target Audience: HPS Project Advisor (Encadrant)
- Generated By: Antigravity AI Coding Assistant
- Core Modules Covered: LangGraph Engine, Hybrid Session Ingestion, Spec-Log Correlation
- Execution Scope: Trace parser limits, deterministic regex matches, LLM cognitive roles
- Presentation Date: August 10, 2026

1. System Architecture Overview

The **Log Analysis Agent** in HPS ComplianceVerifier is built as an agentic validation assistant powered by **LangGraph**. Instead of sending raw, unprocessed transaction data directly to an LLM, the system processes uploads through a multi-stage validation graph. The architecture separates raw data manipulation (deterministic parser tasks) from cognitive synthesis (done by the LLM). This hybrid approach provides 100% accuracy for statistical queries (like occurrences count) while leveraging generative AI for complex diagnostics, error explanation, and recommendations.

Component Role	Tech Stack	Core Responsibility
Workflow Graph	LangGraph StateGraph	Manages conversation turns, state variables, and execution routing.
RAG Query Router	PgVector / PostgreSQL	Indexes persistent and session specifications, querying relevant chunks.
Deterministic Engine	Python / Regex patterns	Performs counts, error classification, and command response map resolution.
Cognitive Synthesizer	Gemini 3.5 Flash	Generates user-facing diagnostic guidance, summaries, and troubleshooting steps.

2. Ingestion Pipeline: What Happens Upon Upload

When a user uploads a document to the ComplianceVerifier session, the processing path splits based on the file format:

A. Uploading a Trace File (.txt / monetic log):

- Session Isolation:** The file is written to a unique folder bound to the user's ``session_id``.
- Transaction Chunking:** The system splits the trace by searching for the separator ``=== Transaction ``. This maps each transaction block individually.
- Deterministic Indexing:** Python counts occurrences of key transaction elements (e.g., command codes like EC, response codes like ED, STAN numbers, or error statuses like NOK/alertes) on the *entire raw file content*.
- RAG Spec Matching:** In parallel, the backend searches the global database for specifications corresponding to the detected command codes. If ``EC`` is found, the backend retrieves specs for ``EC`` and ``ED`` and appends them to the context to help the LLM explain logs, even if no manual was uploaded in this session.
- Intelligent Context Filtering:** To prevent token pollution, if the trace exceeds 80,000 characters, a filter extracts only transactions containing query keywords or failures/errors. It never truncates inside a transaction block.

B. Uploading a Specification PDF (RAG upload):

- PDF Text Extraction:** The backend extracts text using ReportLab/pdfplumber wrappers.
- Recursive Splitting:** The text is divided into chunks (typically 1500 chars with 200 overlap).

3. **Metadata Enrichment:** Each chunk is automatically scanned for command/response pairs (e.g. EC/ED), response values (Value 'B9'), and tags like `contains\_error\_table`.
4. **Session Vector Embedding:** The chunks are embedded using Google embeddings and stored in a temporary session vector table. The RAG engine will restrict search lookups strictly to this file hash when answering session-specific queries.

3. What Could Fail & System Limits

To ensure transparency in front of your advisor, it is critical to present what the system cannot do or where it might fail:

• **Trace Format Violations:**

If the monetic trace is not formatted with standard transaction headers (e.g., missing ``=== Transaction ``), the split algorithm cannot segment it. The system falls back to simple character truncation at the character limit, which can cut transactions in half and degrade the LLM's understanding.

• **Extreme Token Saturation:**

If the log file has hundreds of distinct transactions, all containing errors, the filtering logic might still exceed the 80,000 characters limit. In this case, some transactions are omitted from the LLM context (though a clear warning is printed to the user, and the global counts computed deterministically remain 100% accurate).

• **RAG Segmented Tables:**

If a critical error code table is split exactly across two PDF pages, the RAG chunker might split the table. The LLM might only receive half of the table, leading to omissions unless semantic overlap parameters capture both chunks.

• **Bilingual Ambiguities:**

If the uploaded PDF specifications are in English, but the user queries in French with non-standard monetic translations, pure semantic search might yield low-relevance results. Exact keyword matching rules are deployed to mitigate this by searching for standard code tokens first.

4. Division of Labor: Hardcoded Logic vs. LLM Cognitive Tasks

One of the biggest strengths of this project is that it does not treat the LLM as a database. Here is the strict division of labor:

Deterministic / Hardcoded Logic (Python)	Cognitive / Generative Logic (LLM)
Sub-string Counting: Executing exact <code>`count(token)`</code> searches on the raw text. The LLM is NEVER asked to count occurrences since LLMs are poor at counting tokens in long windows.	Synthesis & Diagnostics: Taking the exact count of errors, reading the relevant RAG spec chunks, and explaining <i>why</i> the error happened.
HSM Code Mapping: Translating response codes to command codes via a static dictionary mapping (<code>`command_response_map.json`</code> linking EC ↔ ED).	Error Code Translation: Reading a chunk showing code <code>`15`</code> and translating it into domain-specific guidance (e.g., 'Check key parity or DUKPT LMK parity').
Token & Metadata Filtering: Pre-filtering the raw content blocks using exact regex keywords (alert, nok, error, -1) to fit constraints.	Context Integration: Reading user chat history, maintaining session tone, and generating coherent explanations in the requested language (French/English).

5. Advisor Presentation Cheat Sheet (Talking Points)

Use these talking points to defend your project design during tomorrow's review:

1. How does the system handle very large trace files?

Answer: We use a hybrid approach. The backend extracts statistics and executes substring counts deterministically on the entire log stream using Python. Then, to stay within token windows and prevent LLM hallucination, we apply a filtering algorithm that extracts only transactions with errors, alerts, or queries. This filters a 2MB log down to relevant transaction packets before passing it to the LLM.

2. Why is pgVector utilized instead of querying the LLM directly on the PDF?

Answer: PDFs are too large (e.g., 746 pages) to pass as direct prompts. We split the PDFs into chunks, index them into a Postgres database using pgvector, and execute similarity searches. For session uploads, we isolate vector store queries to the specific file hash, ensuring users only retrieve information from their uploaded files.

3. What happens if a session terminates or a new user logs in?

Answer: The database stores session chunks mapped to a unique `session\_id`. Files are held in temporary storage. Upon session deletion or timeout, the temporary files are cleared and the database rows matching the `session\_id` are purged, preserving security and isolation.

End of presentation guide. ComplianceVerifier System Engineering. Confidential.